PROBABILITY AND STATISTICS SEMESTER PROJECT | REVIEW 2

A STATISTICAL COMPARISON OF HUMAN WRITTEN AND AI-GENERATED CODE ACROSS PROGRAMMING LANGUAGES

A preliminary study of code length and comment characteristics.

By Akanksha Gupta (25BAI0033) and **Bhargavi Panwar** (25BDS0081) from SCOPE

Faculty Guide: Professor Juhi Baroi.

REVIEW 1 SUMMARY

Dataset

- Total 10,000 code samples are present out of which 5,000 are hand written and 5,000 are AI generated
- Languages used are C, C++, Java and Python.

Analysis Performed

- Descriptive statistics
- Welch's t-tests and 95% confidence intervals
- Chi-square test
- Initial visualisations

Key Findings

- Human code had a higher mean line count: **29.59 vs 25.54**
- AI code had a higher mean character count: **678.63 vs 557.86**
- Both pooled comparisons were statistically significant, with small effect sizes.

Next Step: Compare Human and AI code separately for each programming language

STATISTICAL METHODS USED

1. Welch's t-test
Compares the average values of Human and AI code without assuming equal variances between the two groups.

2. 95% Confidence Interval (CI)
Provides a range of plausible values for the true difference between the group means.

3. Effect Size (Cohen's d)
Measures how large the difference between Human and AI code is, not just whether it is statistically significant.

4. Holm Correction
Adjusts p-values when multiple statistical tests are performed, helping control false positive results.

5. Wilcoxon Rank-Sum Test
A rank-based test used as a robustness check when comparing the distributions of Human and AI code measurements.

Key Statistical Formulas

1 Welch's Independent Samples t-test

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

- Compares the mean code measurements of AI-generated and human-written code (two independent groups).
- Does not assume equal variances between the two groups.
- Used to test if the difference in means is statistically significant.

2 Cohen's d (Effect Size)

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{pooled}}}$$

$$s_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

- Measures the standardized magnitude and direction of the difference between the two group means.
- Larger $|d|$ indicates a greater difference; the sign indicates the direction (positive if group 1 has a higher mean).
- Helps assess practical significance beyond just the p-value.

3 Holm Correction (Multiple Testing)

$$p_{(i)} \leq \frac{\alpha}{m - i + 1}$$

- Adjusts p-values when performing multiple hypothesis tests.
- Sort the p-values in ascending order: $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$.
- Compare each $p_{(i)}$ with $\frac{\alpha}{m-i+1}$ (where m is the total number of tests and i is the rank of the p-value).
- Controls the family-wise error rate (probability of one or more false positives) across all tests.

4 Wilcoxon Rank-Sum Test (Mann–Whitney U Test)

$$U_1 = R_1 - \frac{n_1(n_1 + 1)}{2}$$

- Compares two independent groups using ranks rather than directly comparing means.
- R_1 is the sum of ranks for group 1 and n_1 is its sample size.
- Provides a non-parametric robustness check (does not assume normality).
- A small p-value indicates a significant difference in the distributions of the two groups.

Notation and Variables

- $\bar{x}_1, \bar{x}_2$: Sample means of the two groups (e.g., AI-generated and human-written code)
- s_1, s_2 : Sample standard deviations of the two groups
- n_1, n_2 : Sample sizes of the two groups
- s_{pooled} : Pooled standard deviation (used in Cohen's d)
- $p_{(i)}$: i-th smallest p-value after sorting in ascending order
- α : Significance level (commonly set to 0.05)
- m : Total number of hypothesis tests
- i : Rank of a p-value ($i = 1, 2, \ldots, m$)
- R_1 : Sum of ranks for group 1 (in Wilcoxon rank-sum test)
- U_1 : Mann–Whitney U statistic for group 1
- d : Effect size (Cohen's d)

LANGUAGE-WISE DESCRIPTIVE ANALYSIS

Objective

Descriptive statistics were calculated separately for C, C++, Java, and Python to identify differences between human and AI generated codes.

Key observations

C: AI-generated code had higher mean line and character counts.

C++: Human-written code had a higher mean line count, while mean character counts were similar.

Java: AI-generated code had higher mean character counts, while human-written code had a higher mean line count.

Python: AI-generated code had higher mean character counts, while human-written code had a slightly higher mean line count.

Purpose: To identify language-specific patterns before interpreting the hypothesis tests.

Language	Mean lines (AI / Human)	Mean characters (AI / Human)
C	27.57 / 20.57	654.73 / 294.85
C++	28.94 / 38.26	717.18 / 704.22
Java	29.01 / 33.59	855.07 / 707.56
Python	18.22 / 20.10	504.55 / 361.61

```
# STEP 7: DESCRIPTIVE STATISTICS BY LANGUAGE AND ORIGIN

# Mean and median line count
aggregate(
  Line_Count ~ Language + Generated,
  data = mydata,
  FUN = mean
)

aggregate(
  Line_Count ~ Language + Generated,
  data = mydata,
  FUN = median
)

# Mean and median character count
aggregate(
  Character_Count ~ Language + Generated,
  data = mydata,
  FUN = mean
)

aggregate(
  Character_Count ~ Language + Generated,
  data = mydata,
  FUN = median
)
```

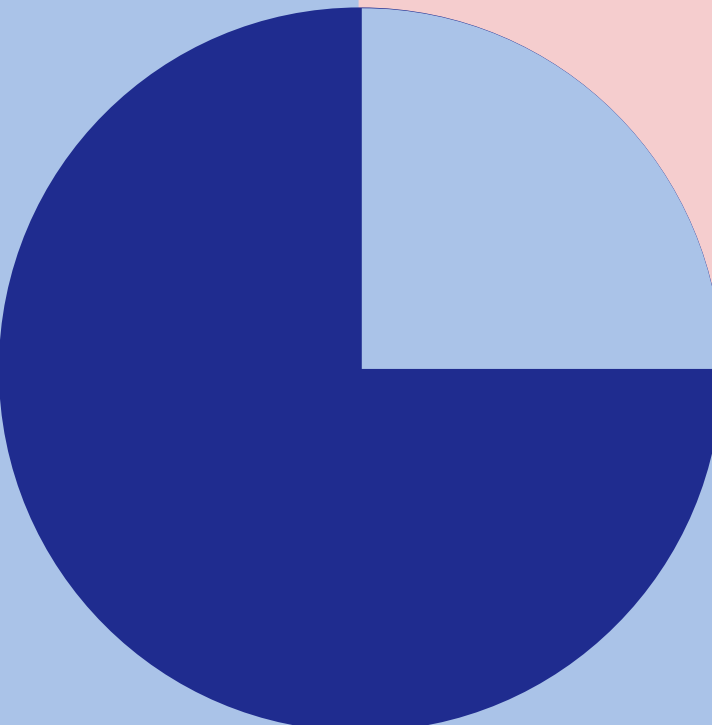
```
> # Mean and median line count
> aggregate(
+   Line_Count ~ Language + Generated,
+   data = mydata,
+   FUN = mean
+ )
  Language Generated Line_Count
1        C         AI   27.57128
2      C++         AI   28.93776
3     Java         AI   29.00896
4   Python         AI   18.22026
5        C      Human   20.56567
6      C++      Human   38.26075
7     Java      Human   33.59315
8   Python      Human   20.09528
>
> aggregate(
+   Line_Count ~ Language + Generated,
+   data = mydata,
+   FUN = median
+ )
  Language Generated Line_Count
1        C         AI         24
2      C++         AI         24
3     Java         AI         24
4   Python         AI         14
5        C      Human         18
6      C++      Human         30
7     Java      Human         25
8   Python      Human         16
>
> # Mean and median character count
> aggregate(
+   Character_Count ~ Language + Generated,
+   data = mydata,
+   FUN = mean
+ )
  Language Generated Character_Count
1        C         AI        654.7273
```

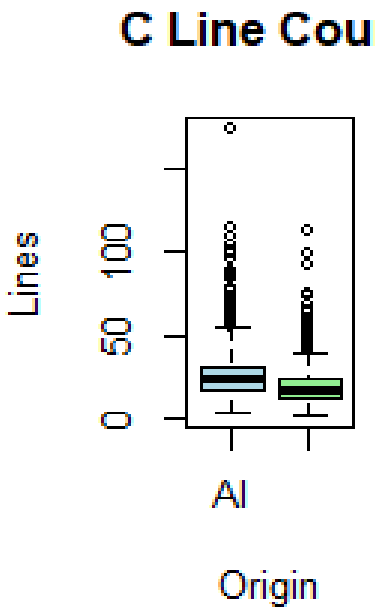
```
# Mean and median character count
aggregate(
  Character_Count ~ Language + Generated,
  data = mydata,
  FUN = mean
)
Language Generated Character_Count
       C         AI        654.7273
     C++         AI        717.1769
    Java         AI        855.0701
  Python         AI        504.5500
       C      Human        294.8492
     C++      Human        704.2220
    Java      Human        707.5564
  Python      Human        361.6139

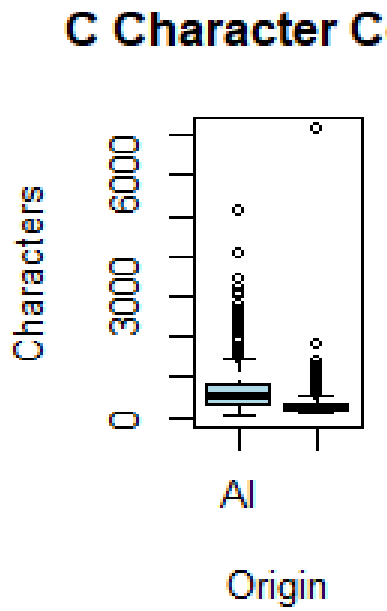
aggregate(
  Character_Count ~ Language + Generated,
  data = mydata,
  FUN = median
)
Language Generated Character_Count
       C         AI            535.0
     C++         AI            557.0
    Java         AI            686.5
  Python         AI            385.0
       C      Human            235.0
     C++      Human            492.0
    Java      Human            515.0
  Python      Human            278.0
```

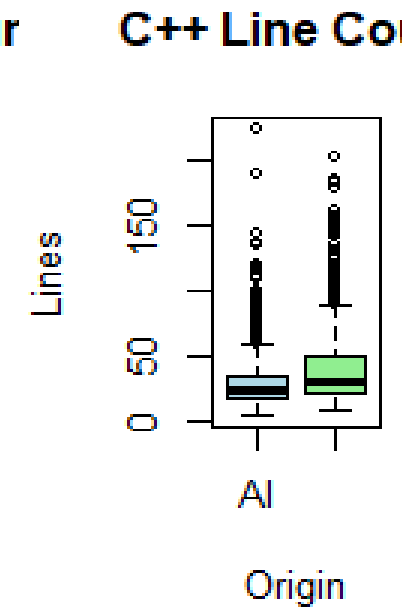
GRAPHS

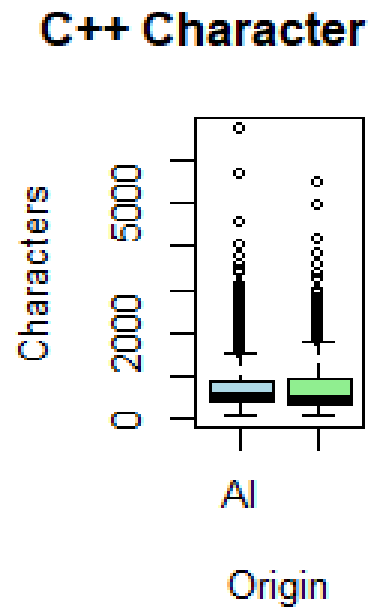
"Boxplots compare the median, spread, and outliers of AI generated and human written code across four programming languages.

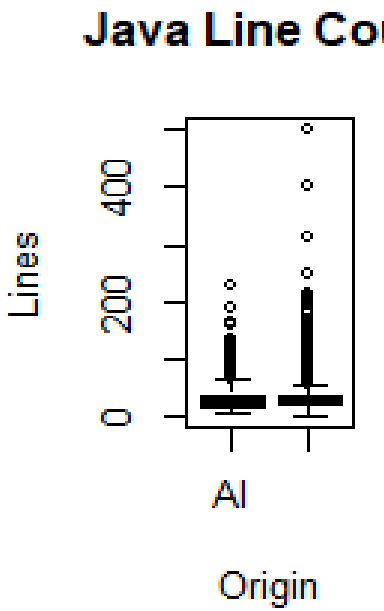


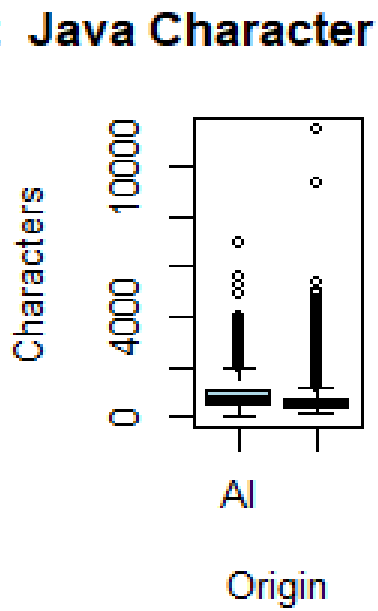


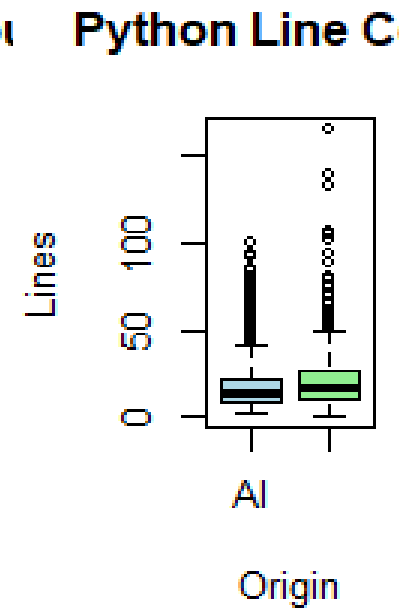


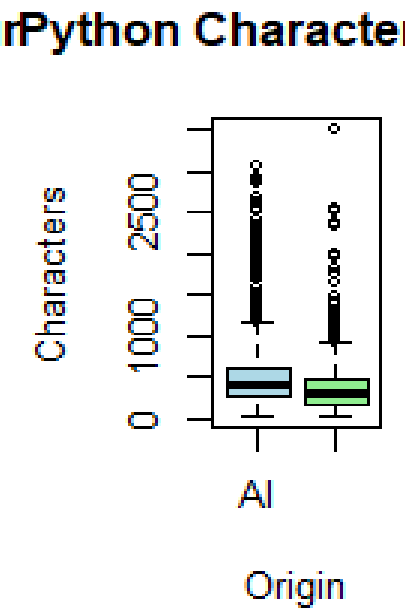












WELCH'S T TEST RESULT

Welch's t-test was used to compare the mean code length of AI generated and human written samples within each programming language.

Hypotheses

- **H_0:** The mean code length is equal for AI generated and human written samples.
- **H_1:** The mean code length differs between the two groups.

Key findings from the preliminary analysis

- C: Significant differences in both line count and character count.
- C++: Significant difference in line count, but not in character count after Holm correction.
- Java: Significant differences in both measurements.
- Python: Significant differences in both measurements.

Statistical criteria: 5% significance level, 95% confidence intervals, and Holm correction for eight comparisons.

```r
# Compare AI and Human within one language
run_test <- function(variable, lang) {

  subset_data <- mydata[mydata$Language == lang, ]

  ai_values <- subset_data[
    subset_data$Generated == "AI", variable
  ]

  human_values <- subset_data[
    subset_data$Generated == "Human", variable
  ]

  result <- t.test(ai_values, human_values,
                   var.equal = FALSE)

  data.frame(
    Language = lang,
    Measurement = variable,
    AI_Mean = mean(ai_values),
    Human_Mean = mean(human_values),
    Difference = mean(ai_values) - mean(human_values),
    t_statistic = unname(result$statistic),
    df = unname(result$parameter),
    p_value = result$p.value,
    CI_Lower = result$conf.int[1],
    CI_Upper = result$conf.int[2]
  )
}
```

Language	Measurement	Holm-adjusted p-value	Conclusion
C	Line count	8.93×10^{-23}	Significant
C	Character count	6.96×10^{-72}	Significant
C++	Line count	3.08×10^{-24}	Significant
C++	Character count	0.560	Not significant
Java	Line count	8.71×10^{-6}	Significant
Java	Character count	1.02×10^{-8}	Significant
Python	Line count	0.00218	Significant
Python	Character count	1.67×10^{-23}	Significant

Significance level: $\alpha = 0.05$
Takeaway: Seven of the eight comparisons were statistically significant after Holm correction; C++ character count was the exception.

```
> run_test <- function(variable, lang) {
+
+   subset_data <- mydata[mydata$Language == lang, ]
+
+   ai_values <- subset_data[
+     subset_data$Generated == "AI", variable
+   ]
+
+   human_values <- subset_data[
+     subset_data$Generated == "Human", variable
+   ]
+
+   result <- t.test(ai_values, human_values,
+                    var.equal = FALSE)
+
+   data.frame(
+     Language = lang,
+     Measurement = variable,
+     AI_Mean = mean(ai_values),
+     Human_Mean = mean(human_values),
+     Difference = mean(ai_values) - mean(human_values),
+     t_statistic = unname(result$statistic),
+     df = unname(result$parameter),
+     p_value = result$p.value,
+     CI_Lower = result$conf.int[1],
+     CI_Upper = result$conf.int[2]
+   )
+ }
```

"Welch's t-tests compare group means; Holm correction controls the family-wise error rate across eight comparisons."

EFFECTIVE SIZE ANALYSIS

Cohen's *d* was used to estimate the magnitude of the differences between AI-generated and human-written code.

Key findings from the preliminary analysis:

C: Large effect for character count ((≈0.87)); small-to-moderate effect for line count ((≈0.47)).

C++: Small-to-moderate effect for line count ((≈-0.40)); negligible effect for character count ((≈ 0.02)).

Java: Small effects for both measurements.

Python: Small-to-moderate effect for character count ((≈0.39)); small effect for line count.

Interpretation: Statistical significance does not necessarily imply a large or practically important difference.

```r
# STEP 9: CALCULATE COHEN'S D

calculate_d <- function(variable, lang) {

  subset_data <- mydata[mydata$Language == lang, ]

  ai_values <- subset_data[
    subset_data$Generated == "AI", variable
  ]

  human_values <- subset_data[
    subset_data$Generated == "Human", variable
  ]

  n_ai <- length(ai_values)
  n_human <- length(human_values)

  pooled_sd <- sqrt(
    ((n_ai - 1) * var(ai_values) +
       (n_human - 1) * var(human_values)) /
      (n_ai + n_human - 2)
  )

  (mean(ai_values) - mean(human_values)) / pooled_sd
}
```


```r
# STEP 9: CALCULATE COHEN'S D

calculate_d <- function(variable, lang) {

  subset_data <- mydata[mydata$Language == lang, ]

  ai_values <- subset_data[
    subset_data$Generated == "AI", variable
  ]

  human_values <- subset_data[
    subset_data$Generated == "Human", variable
  ]

  n_ai <- length(ai_values)
  n_human <- length(human_values)

  pooled_sd <- sqrt(
    ((n_ai - 1) * var(ai_values) +
       (n_human - 1) * var(human_values)) /
      (n_ai + n_human - 2)
  )

  (mean(ai_values) - mean(human_values)) / pooled_sd
}
```

ROBUSTNESS CHECK: WILCOXON TEST

- The Wilcoxon rank-sum test was used as a non parametric comparison of AI-generated and human-written code.
- It compares the distributions using ranks rather than relying on the same assumptions as a t-test.
- Its results provide a robustness check alongside Welch's t-test.
- Differences should be interpreted using adjusted p values where multiple comparisons are performed.
- **Conclusion:** Comparing both tests helps assess whether the observed differences are sensitive to the choice of statistical method.

Metric	AI-generated	Human-written
Mean comment count	1.1184	0.6092
Median comment count	0	0

Wilcoxon rank-sum test: ($p < 2.2 \times 10^{-16}$)
AI-generated code had a higher mean comment value than human-written code.
However, more comments do not necessarily indicate better documentation.

2. Limitations

- **Approximate counting:** The method may misclassify comment markers or miss some comments.
- **Sample independence:** Only three problem IDs contained both Human and AI samples; repeated IDs may introduce dependence.
- **Measurement validation:** Line and comment counts require further validation.
- **Interpretation:** Code length and comment count alone cannot establish code quality or correctness.

Conclusion: The results indicate differences in counted comments, but should be interpreted cautiously due to measurement and dataset limitations.

OVERALL FINDINGS & FUTURE WORK

- **Key Findings**
- Differences in code length varied across C, C++, Java, and Python.
- AI-generated code generally had higher mean character counts in the preliminary analysis.
- Seven of eight Welch's t-test comparisons were statistically significant after Holm correction.
- Comment-count analysis showed a higher mean for AI-generated code.
- **Limitations**
- Code length and comment count do not directly measure code quality.
- Repeated problem IDs may affect the independence assumption.
- Code-count measurements require validation.
- **Future Work**
- Validate line and comment-count calculations.
- Investigate the effect of programming language and code origin together.
- Extend the analysis and finalize conclusions using validated measurements.

THANK YOU